

懂吃 DongChi

技术文档

检索增强的饮食决策系统——让 AI 只做它擅长的事

赛事　第九届传智杯全国大学生数智创新与 AI 应用大赛

赛道　AI 创新应用挑战赛（个人赛）

平台　HarmonyOS 6.0.2（API 22）· ArkTS / ArkUI

应用包名　com.example.dongchi

版本　v1.1.0

目　录

一、项目概述

1.1 问题定义　1.2 解决方案　1.3 核心主张

二、技术架构

2.1 分层结构　2.2 技术选型与理由

三、AI 能力融合（核心）

3.1 为什么不能一次裸调用　3.2 检索增强 + 受约束重排　3.3 数值与语义分离

3.4 提示词工程　3.5 输出解析的健壮性　3.6 降级策略

四、工程实现

4.1 性能　4.2 正确性　4.3 并发与资源　4.4 凭据与安全

五、测试与评估

5.1 单元测试　5.2 测试设计思路　5.3 端到端实测　5.4 评估结论

六、已知局限与后续规划

七、附录

7.1 构建与运行　7.2 代码规模　7.3 关键文件索引　7.4 复现与评审检查清单

一、项目概述

1.1 问题定义

“今天吃什么”是每个人每天要面对三次的高频决策。真正的难点并不在于“不知道有哪些菜”，而在于约束条件下的取舍：时间够不够、预算够不够、冰箱里还剩什么、有没有忌口，以及一个很少被其他工具回答的问题——自己做和点外卖，哪个更划算。

这类决策的痛点有三个特征，也正是本项目的切入点：

- **高频且刚性：**一天三次，无法回避，且每次都伴随决策摩擦成本。
- **多约束耦合：**口味、时间、预算、现有食材、忌口彼此制约，靠“随机转盘”无法收敛。
- **缺少成本视角：**多数推荐工具只回答“吃什么”，不回答“怎么吃更省”，而后者直接影响长期支出。

1.2 解决方案

懂吃是一套决策系统，而非推荐工具。它把一次饮食决策拆成四个可验证的环节：

```
三问渐进式收敛条件 → AI 在合规候选内重排并解释 → 自己做 vs 点外卖对比
                                                          |
                                                  冰箱食材管理 + 购物清单闭环
```

其中“自己做 vs 点外卖”的对比是本项目区别于同类应用的差异化能力：对每道推荐菜同时给出自制耗时与食材成本、外卖配送时长与价格，并精确指出家中已有哪些所需食材、还缺哪几样，从而把“吃什么”延伸到“怎么吃更划算”。

1.3 核心主张

硬约束由确定性代码保证，生成式模型只负责它真正擅长的语义理解与解释。

这条原则贯穿整个 AI 设计。第三章将说明它如何从一个具体的工程问题推导出来，并如何消除了“AI 推荐忌口菜”“AI 编造不存在的菜”“AI 编造价格与耗时”这三类缺陷。

二、技术架构

2.1 分层结构

应用采用四层结构，层间职责单一，便于替换与测试：

层次	职责	主要实现
表现层	页面渲染、交互与状态	ArkUI，15 个页面
决策编排层	把用户条件转成推荐结果，串联检索与重排	RecommendService
AI 能力层	提示词构造、结果解析与校验、降级	CloudAiService
数据层	本地持久化与内置数据	DbService（relationalStore）、500 道菜品库、498 道详情

2.2 技术选型与理由

技术点	选型	理由
AI 接入	阿里云百炼（OpenAI 兼容）	新用户每模型约 100 万 Token 免费额度，学生可零成本开发；兼容格式便于更换供应商
默认模型	qwen-flash	在免费额度覆盖范围内，延迟低，适合短输出任务
本地存储	relationalStore（SQLite）	决策记录、收藏、冰箱需关系查询与去重，不适合 KV
配置存储	Preferences	轻量键值，用于 AI 密钥与开关
界面框架	ArkUI 声明式	HarmonyOS 原生支持，便于统一沉浸式适配

三、AI 能力融合（核心）

3.1 为什么不能“把条件丢给大模型”

项目早期实现是典型的“一次裸调用”：把用户条件拼成提示词，让模型直接输出三道菜的 JSON。实测暴露三类问题：

问题	表现	后果
幻觉	模型推荐菜品库中根本不存在的菜	无法展示详情、无法核算成本，链路断裂
约束违反	提示词已写明“不吃辣”，仍可能推荐麻辣香锅	自然语言约束对生成模型不是硬保证

问题	表现	后果
事实失真	模型自行编造烹饪耗时与价格	与本地数据不一致，用户看到的事实信息不可信

根因归因：这三类问题看似独立，但共同指向同一个结构缺陷——**把“约束求解”和“语义理解”两类完全不同的任务交给了同一个组件。**约束求解需要确定性保证，而语义理解需要生成能力；把两者混在一起，就等于用概率性系统去承担确定性责任。

3.2 检索增强 + 受约束重排

据此把推荐拆成三段：把不可信的生成空间收敛成可信的候选集，再让模型在其中做语义排序，最后用纯函数校验兜住一切意外。

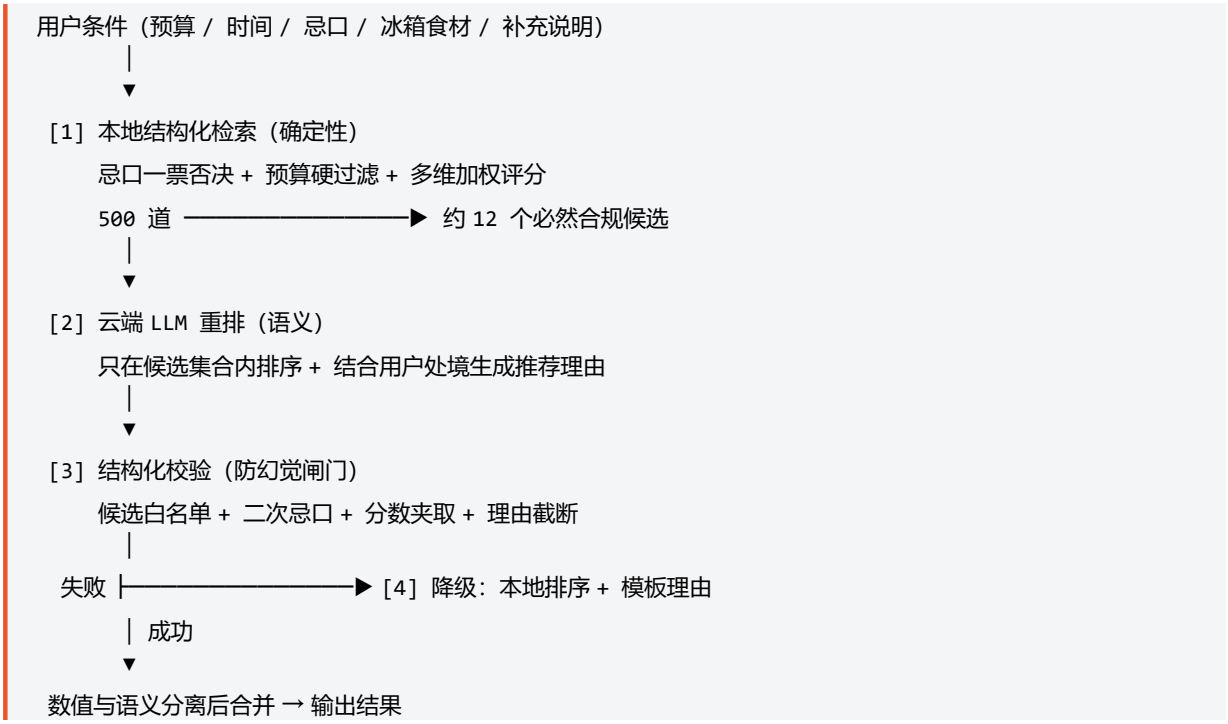


图 3-1　三段式推荐管道：约束求解（1、3）与语义理解（2）分离

三段各自的职责边界如下：

- **第一段（本地检索）：**忌口、预算属于硬约束，由确定性代码执行，任何后续环节都不允许绕过。此段同时产出候选的量化特征（自制/外卖耗时与成本、所需食材及“家中已有 N/M 样”）。
- **第二段（云端重排）：**模型只在候选集合内排序。由于候选已经过合规过滤，模型从架构上就不可能推荐忌口菜或库外菜；它只需发挥语义能力，结合“两个人吃”“今天不想动手”这类结构化条件无法表达的软信息给出理由。
- **第三段（校验）：**把模型输出当作不可信输入处理。菜名必须在候选白名单内，忌口二次扫描，分数夹取到 0–100，理由截断限长；任一项不通过即判为无效。

3.3 数值与语义分离

这是整个 AI 设计中最关键的一条约束：模型只决定“排序”和“推荐理由”，所有时间、成本、食材是否齐备等事实信息一律取自本地数据。

```
// mergeRerankWithLocal：只采纳模型的排序与理由
const dish = cloneLocalDish(local);        // 时间 / 成本 / 食材来自本地
dish.reason = item.reason;                 // 仅理由来自模型
dish.hasIngredients = hasRequiredIngredients(...);  // 由本地规则判定
```

这样即使模型对价格或耗时的判断有偏差，也不会污染用户看到的事实信息。模型的能力被限定在“解释与取舍”，而不是“陈述事实”。

3.4 提示词工程

系统提示词明确四条硬规则，避免模型自由发挥：

```
1. 只能从候选清单里选，禁止引入清单之外的任何菜名，禁止编造菜名。
2. 推荐理由要结合用户的具体处境（几人吃、冰箱里已有什么、时间是否紧张、
   预算是否敏感），不要说“营养美味”“强烈推荐”这类空话。
3. 理由不超过 30 个字，直接给出判断依据，不要重复菜名。
4. 严格输出 JSON，不要输出任何解释文字、不要用 markdown 代码块包裹。
```

用户提示词把结构化条件自然语言化，并附带候选的量化特征，让模型有充分依据做取舍：

```
【用户条件】
- 口味方向：家常清淡    - 时间预算：30分钟
- 花费预算：20元以内    - 忌口限制：不吃辣
- 家里现有食材：鸡蛋、番茄、大米
- 用户补充说明：两个人吃，今天不太想动手
- 历史偏好：收藏过番茄炒蛋、蛋炒饭；最常选的口味方向是家常清淡

【候选菜品清单】（只能从这里选，共 8 道）
1. 番茄炒蛋 | 家常清淡 | 自己做约10分钟/8元 | 外卖约25分钟/18元 | 需要：番茄、鸡蛋（家中已有 2/2 样）
2. 清蒸鲈鱼 | 家常清淡 | 自己做约25分钟/28元 | 外卖约30分钟/45元 | 需要：鲈鱼、姜、葱（家中已有 0/3 样）
...
```

“补充说明”入口的设计动机：结构化条件无法表达“两个人吃”“今天不想动手”这类软信息，而这恰恰是模型能发挥价值的地方。这是把 AI 从“打分器”变成“顾问”的关键入口。

3.5 输出解析的健壮性

真实模型经常不完全遵守“只输出 JSON”。解析器采用括号配对扫描而非正则：

- 剥离 markdown 围栏后，从第一个 { 开始逐字符扫描；
- 维护字符串状态与转义状态，跳过字符串内部的花括号；
- 遇到深度归零的 } 即返回，天然支持嵌套对象。

相比常见的贪婪正则 /\{[\s\S]*\}/，该实现能正确处理“JSON + 前后解释文字”与嵌套对象；贪婪匹配在模型附带解释时会一次性吞掉过多内容，导致解析必然失败。

3.6 降级策略

降级不是兜底补丁，而是设计的一部分：本地引擎本身就是完整可用的推荐器，云端 AI 提供的是“更贴合的排序与解释”这一增量价值。

触发条件	行为
未配置 API Key	纯本地推荐，设置页提示填写
用户在设置中关闭 AI	纯本地推荐
网络异常 / 超时	回退本地排序，并向用户说明原因
免费额度用尽（HTTP 403）	回退本地排序，错误翻译为可懂文案
模型输出无法解析	自动重试一次（温度降至 0.1），仍失败则回退
校验后无有效结果	回退本地排序

四、工程实现

4.1 性能

推荐链路涉及 500 道菜的过滤与打分，容易在 UI 线程形成卡顿。主要优化如下：

优化项	优化前	优化后
候选克隆	每次推荐全量克隆约 500 道菜	只克隆最终选中的 3 道
权重计算	在 500 次循环内重复计算	循环外计算一次
结果挑选	两次 filter 构造中间数组 + 全量排序	单次线性扫描
决策记录渲染	每帧对每条记录 JSON.parse	加载时一次性解析并缓存
购物清单图标	90 项映射表每次调用重建	提升为模块常量
仪表盘统计	@State 数组逐次 push + sort	本地构建后单次赋值

4.2 正确性

- **忌口双重校验：**本地字段过滤（isSpicy / isSeafood / isVegetarian）+ 云端返回后菜名二次扫描。

- **演示数据隔离**：AI 与本地引擎均失败时的硬编码兜底菜不写入决策记录，避免污染仪表盘统计。
- **决策去重**：同一次决策内的“换一批”不重复写库。
- **画像时效**：决策新增、记录删除、收藏变化均会使偏好画像缓存失效。

4.3 并发与资源

- **TTS 回调归属**：监听器校验 requestId，丢弃过期事件，修复“图标显示未播报但声音仍在响”的状态错乱。
- **TTS 初始化**：使用 in-flight Promise 复用初始化任务，防止并发初始化泄漏引擎实例。
- **页面定时器**：在 aboutToDisappear 中清理，避免回调打到已销毁组件。
- **一次性状态**：页面级选择态在离开时复位，避免下次进入残留。

4.4 凭据与安全

仓库不包含任何签名口令、证书文件或 API Key，具体处置如下：

凭据	存放位置	是否入库
签名口令	signing/signing-config.local.json	否（.gitignore）
签名证书 .p12 / .p7b / .cer	项目外，由上面文件指向	否（.gitignore）
百炼 API Key	rawfile/ai-config.local.json 或应用 Preferences	否（.gitignore）
AI 密钥模板	rawfile/ai-config.local.json.example	是
签名配置模板	signing/signing-config.example.json	是

build-profile.json5 必须入库（承载模块结构与编译配置），因此其中 signingConfigs 留空；需要产出自用的已签名包时，由 scripts/build-signed.mjs 从本地私有文件临时注入，并在构建结束后无条件还原，避免口令落回工作区。

已知局限：客户端持有密钥始终存在被反编译提取的风险。生产环境应改为服务端代理——仅需把 baseUrl 指向自建接口，检索、重排、校验逻辑无需改动。

五、测试与评估

5.1 单元测试

AI 应用的测试难点在于模型输出不确定。本项目的应对是把不确定性挡在校验层之外：模型输出先经过一个纯函数闸门，因此可脱离网络、用构造输入覆盖全部边界情况。

用例	验证目标
防幻觉	候选集合之外的菜名必须被丢弃
忌口拦截	即使模型返回违规菜名也必须被拦下
数量截断	结果上限 3 条
分数夹取	越界分数夹到 0–100
理由截断	超长理由截断至 40 字
理由兜底	空理由填充中性文案
非法输入	非 JSON 返回判为无效，触发降级
重复菜名	只保留一条
JSON 提取	穿透 markdown 围栏与前后解释文字
嵌套对象	括号配对不提前截断
字符串花括号	字符串内 {} 不干扰配对

共 12 个用例（含一条数量边界），位于 entry/src/ohosTest/ets/test/AiPipelineTest.ets，已接入测试套件；执行需真机或模拟器。

5.2 测试设计思路

不只做“能跑通”的演示，而是主动构造对抗性输入验证架构假设，例如“幻觉压力测试”：故意只提供白粥、水煮蛋、清炒白菜这类不诱人的候选，检验模型是否会自行越界加菜。这类用例才是架构有效性的直接证据。

5.3 端到端实测

用真实 API 在五个场景下跑通全链路，结果如下（模型 qwen-flash）：

表 5-1　五场景端到端实测结果

场景	目的	耗时	结果
基础场景	链路是否通	1.27s / 869 tokens	通过，理由正确使用“家中已有食材”
严格忌口（不吃辣+素食）	约束是否零泄漏	1.14s	零泄漏，主动避开候选中的辣菜
极紧预算（10 元内）	预算是否硬过滤	0.90s	推荐均落在预算内
幻觉压力测试	候选不诱人时是否越界	0.99s	未越界，仅对给定候选排序

场景	目的	耗时	结果
无效密钥	降级路径是否生效	—	如期返回 401，触发降级

基础场景中模型的真实输出如下（节选）：

```
{"items":[
  {"name":"番茄炒蛋","score":95,"reason":"食材全有，10分钟搞定，最合清淡省事需求"},
  {"name":"蛋炒饭","score":90,"reason":"剩饭+鸡蛋即成，12分钟出餐，省力又饱腹"},
  {"name":"紫菜蛋花汤","score":85,"reason":"仅缺紫菜，8分钟完成，清淡解腻正合适"}
]}
```

这段输出可以佐证两件事：其一，模型确实读取并使用了结构化信息中的“家中已有 2/2 样”，理由不是模板句；其二，它主动避开了候选清单里同样存在、但违反“不吃辣”的麻婆豆腐。

5.4 评估结论

指标	结果
约束违反率	0%（忌口与预算由本地硬过滤保证）
解析一次成功率	5/5 场景首次调用即解析并通过校验
降级可用性	未配置/断网/额度用尽/解析失败均回退本地且推荐非空
单次调用开销	约 0.9–1.3s；提示词 774 tokens，固定前缀可命中上下文缓存

六、已知局限与后续规划

局限	说明	规划
营养数据为区间估算	498 道菜营养值按菜系粗分，仅 7 组区间；页面已标注“仅供参考”	接入公开食物成分数据库逐菜补全
步骤为通用做法	步骤文本去重后约 300 条；页面已标注“通用做法”	逐条替换为真实菜谱
客户端持有密钥	反编译可提取	增加服务端代理，仅替换 baseUrl
使用旧版 router API	编译存在弃用告警	迁移至 Navigation 路由

局限	说明	规划
长列表未虚拟化	收藏/记录/冰箱页使用 Scroll + ForEach	改为 List + LazyForEach

七、附录

7.1 构建与运行

```
# 1) 可选：配置云端 AI（不配置则使用本地引擎，功能完整可用）
cp entry/src/main/resources/rawfile/ai-config.local.json.example \
   entry/src/main/resources/rawfile/ai-config.local.json

# 2) 编译
set DEVECO_SDK_HOME=<DevEco Studio>/sdk
set JAVA_HOME=<DevEco Studio>/jbr
set PATH=%JAVA_HOME%\bin;%PATH%
hvigorw assembleHap --mode module -p product=default -p buildMode=debug

# 3) 需要已签名包时（从本地私有配置注入，构建后自动还原）
set HVIGOR_BAT=<DevEco Studio>\tools\hvigor\bin\hvigorw.bat
node scripts/build-signed.mjs
```

7.2 代码规模

模块	规模	说明
页面（ArkUI）	15 个	三问决策、推荐、详情、冰箱、清单、记录、收藏、仪表盘、设置
服务层	8 个	AI 接入、决策编排、数据库、TTS、共享常量、状态传递
内置数据	6 个	500 道菜品（4 分片）+ 498 道详情
测试	2 个	测试套件 + AI 链路用例（12 个）

7.3 关键文件索引

文件	职责
service/CloudAiService.ets	AI 能力层：提示词、重排、解析校验、降级、密钥管理
service/RecommendService.ets	决策编排：三段式管道与本地推荐引擎

文件	职责
service/DbService.ets	relationalStore 数据访问
service/LocalDishes.ets	500 道菜品库合并入口
pages/AiSettings.ets	AI 能力设置：密钥、模型、连通性测试
pages/Recommend.ets	推荐结果展示与补充说明输入
scripts/build-signed.mjs	本地签名注入与构建（构建后自动还原）
ohosTest/test/AiPipelineTest.ets	AI 校验闸门单元测试（12 个用例）

7.4 复现与评审检查清单

按以下顺序可在干净环境中复现本项目全部可验证行为：

步骤	操作	预期结果
1	安装 DevEco Studio 6.1+ 与 HarmonyOS SDK，克隆仓库	目录下无任何密钥文件，build-profile.json5 的 signingConfigs 为空
2	不配置任何密钥，直接编译	构建成功，产出 entry-default-unsigned.hap
3	App 内「我的 → AI 能力设置」填入百炼 API Key，或放置 rawfile/ai-config.local.json	推荐结果出现结合“家中已有食材”的个性化理由
4	关闭 AI 开关后重新推荐	推荐仍可用，理由退化为通用文案（降级路径生效）
5	填入无效 API Key 后推荐	提示密钥无效并自动回退本地排序，不崩溃
6	运行 ohosTest 中的 AiPipelineTest	12 个用例通过，覆盖防幻觉、忌口拦截、分数夹取等
7	执行 node scripts/build-signed.mjs	产出已签名 HAP；结束后 build-profile.json5 已自动还原
8	在仓库中检索 storePassword / apiKey 实际值	仅命中示例模板中的占位符，无真实凭据

对照赛道要求，本项目提交物对应关系为：代码仓库（含细粒度提交历史）对应本仓库 git 记录；AI 对话历史 Prompt 快照见 AI-PROMPTS.md；复现指南见 README.md；技术文档即本文档；演示视频另行提交。